# BHTTP/1 - Binary File Transfer Protocol

Network Architecture Project | Protocol specification | 8 October 2026

## 1. Scope and transport

This document is sufficient to implement either peer independently. MUST, MUST NOT and MAY express requirements. BHTTP/1 carries GET-like file requests, status, headers and arbitrary file bytes over TCP. It is a course protocol inspired by HTTP semantics; it is not wire-compatible with HTTP/1, HTTP/2 or HPACK. Both peers agree on BHTTP/1 before connecting. There is no preface, negotiation, TLS, upgrade, request body, redirect, compression, multiplexing or server push. The default demonstration port is 9000; any agreed TCP port may be used.

A client MUST use one connected TCP socket for the entire session and MUST NOT retry on a new connection. It sends one REQUEST and reads the complete response before sending the next. Request IDs are 1..65535, incrementing and wrapping to 1 after a completed response. A server MUST echo the received ID and MUST keep the connection open after success or recoverable errors. END ends a message, never the connection. The client closes after its last response. The reference server closes after 60 seconds without incoming data; the client uses a 10-second socket timeout. EOF between frames ends the session; EOF within a header or payload is fatal and the receiver closes without attempting resynchronization.

## 2. Fixed-size frame header

Every frame is exactly an 8-byte header followed by Length payload bytes. Integers are unsigned, big-endian (network byte order). There is no padding or checksum; TCP provides reliable ordered delivery. A receiver MUST read exact byte counts: one TCP recv call is not one frame.

| Byte offsets | Field | Width | Meaning |
|---|---|---|---|
| 0..3 | Length | 32 bits | Payload byte count, excluding the 8-byte header |
| 4 | Type | 8 bits | 1 REQUEST; 2 RESPONSE; 3 DATA; all others unknown |
| 5 | Flags | 8 bits | Bit 0 END; bits 1..7 MUST be zero on known types |
| 6..7 | Request ID | 16 bits | Nonzero request correlation; echoed in response/data |

Known-type payloads MUST be at most 65,536 bytes. The sender chunks file data; the reference sender uses at most 16,384 bytes per DATA frame. For an oversized known request-side frame, the server drains its advertised payload in bounded chunks and sends 400, then resumes. A client receiving an oversized known frame fails the session. ID 0 is invalid for requests; a 400 diagnostic MAY echo 0 when that invalid ID was received.

**A receiver meeting a frame type it does not know MUST skip it cleanly.** It consumes exactly Length bytes, including lengths above 65,536, using bounded buffers; ignores its flags and ID; sends no error for that unknown frame; leaves its message state unchanged; and resumes at the next frame header. An unknown frame is legal before or between known frames. No skipping rule can recover a physically truncated frame.

## 3. Payload grammar

Notation: u8/u16 are unsigned 1-/2-byte integers; bytes[n] is exactly n bytes; + means concatenation. All text lengths count encoded bytes, not characters. Text uses strict UTF-8 except header names, which use ASCII.

REQUEST (Type=1, Flags=END): method:u8 + path_length:u16 + path:bytes[path_length] + headers. Only method=1 (GET) is assigned. path_length is 1..4096. All request fields and headers MUST fit one frame. RESPONSE (Type=2): status:u16 + headers. Status MUST be 100..599; the file server emits 200, 400, 403, 404 or 500. DATA (Type=3): raw nonempty body bytes. File bytes are never decoded or transformed.

Each response starts with exactly one RESPONSE. It MUST contain content-length as nonempty ASCII decimal digits, declaring the total body length. For a zero-byte body, RESPONSE MUST set END and no DATA follows. Otherwise RESPONSE flags are zero, followed by one or more DATA frames; only the last DATA sets END. All known response frames MUST echo the outstanding ID. The client rejects unexpected types, reserved flags, duplicate RESPONSE, bad IDs, empty DATA, missing/invalid content-length, overflow, or a final byte count different from content-length. HTTP error status bodies follow this same framing.

# BHTTP/1 - Headers, behavior and rationale



## 4. Compact headers

headers = count:u8 + entry[count], with count 0..32. An indexed-name entry is name_id:u8 + value_length:u16 + value:bytes[value_length]. For name_id=0, insert name_length:u8 + name:bytes[name_length] immediately after name_id. Literal names are 1..255 bytes. Values are 0..65535 bytes of strict UTF-8, without U+0000..U+001F or U+007F. Names MUST be nonempty lowercase ASCII tokens using letters, digits, or ! # $ % & ' * + - . ^ _ ` | ~. Duplicate names, unassigned IDs 11..255, invalid text, truncated fields and extra payload bytes are malformed. Unknown literal header names are valid and ignored if not understood. There is no dynamic table, indexed value or Huffman encoding.

| ID | Name | Typical use | ID | Name | Typical use |
|---|---|---|---|---|---|
| 1 | host | Request authority | 6 | server | Response software |
| 2 | user-agent | Client software | 7 | cache-control | Response cache policy |
| 3 | accept | Accepted media | 8 | last-modified | File modification time |
| 4 | content-type | Body media type | 9 | etag | Weak file metadata validator |
| 5 | content-length | Body byte count | 10 | x-request-id | Decimal frame ID |

The reference client sends IDs 1, 2, 3 and 10. Successful file responses send 4..10; error responses send 4, 5, 6, 7 and 10. Thus all ten indexed names are used. x-request-id, when present, MUST match the decimal frame ID. No request header is mandatory. Requests have no body regardless of their headers. For example, the literal x-test: yes encodes as 01 00 06 78 2d 74 65 73 74 00 03 79 65 73 (count, literal marker, name length/name, value length/value).

## 5. Paths, errors and session behavior

A target MUST start with /, and MUST NOT contain a query or fragment. Decode percent escapes exactly once, rejecting malformed escapes or invalid UTF-8. Reject NUL, other ASCII control characters, DEL and backslash. Reject any decoded . or .. path segment with 403. Resolve the remaining relative path below the configured root, including symlinks, and reject an escape outside that root with 403. Map / to /index.html. A missing file or directory target receives 404; permissions receive 403. The serving root is trusted and must not be changed concurrently by an untrusted local writer; resolution alone is not a defense against local symlink races.

Malformed REQUEST payloads, unsupported methods, bad known-type flags/ID, or known RESPONSE/DATA arriving at the server receive 400 after the full frame is consumed. Preserve the connection so the next valid request can succeed. File open/metadata failures before a response begins receive 500 unless classified as 403/404. If a file changes or an I/O error occurs after response transmission starts, close the connection; never inject another RESPONSE into that body. Every recoverable error has a text/plain; charset=utf-8 body and accurate content-length. The reference bodies are "400 Bad Request\n", "403 Forbidden\n", "404 Not Found\n" and "500 Internal Server Error\n". The client writes body bytes only to stdout, all diagnostics/hexdumps to stderr, exits 1 if any response status is 400..599, exits 2 on local/protocol/transport errors, and otherwise exits 0.

## 6. Why these fields and widths?

32-bit Length gives a directly aligned 4-byte integer and enough skip distance for future frame types; a separate 64 KiB limit bounds allocation for current types. 8-bit Type leaves 253 extension values. 8-bit Flags allows seven future bits; strict validation catches mistakes in current types. 16-bit request IDs cost two bytes and are sufficient for one outstanding request, with reuse only after completion. The resulting header is 8 bytes. END and content-length deliberately overlap: END provides framing, while the length independently catches missing or excess body bytes. A fixed name ID saves repeated header-name bytes; literal lengths make new names unambiguous without shared mutable compression state.

HTTP/2 uses 24-bit Length, 8-bit Type, 8-bit Flags and a reserved bit plus 31-bit Stream Identifier in a 9-byte header [1]. Its large identifier space supports multiplexed streams; this sequential protocol needs less space. HTTP/2's 24-bit length fits its frame design, whereas this project chooses 32 bits for simple aligned parsing. HPACK supports indexed representations and literal fields [2]; BHTTP/1 borrows fixed indexed names plus literal fallback, but deliberately defines its own much smaller encoding. Skipping unknown types preserves frame alignment for compatible extensions; incompatible changes require a separately agreed protocol version/port.

References: [1] RFC 9113, section 4.1: https://www.rfc-editor.org/rfc/rfc9113.html#section-4.1. [2] RFC 7541, sections 6.1-6.2: https://www.rfc-editor.org/rfc/rfc7541.html#section-6.